

# HueVista - Mobile API Guide

Complete API reference for building the HueVista mobile application. This document covers every endpoint, request format, response format, error handling, and authentication flow your mobile app needs.

---

## Table of Contents

1. [Getting Started](#)
  2. [Authentication Overview](#)
  3. [Auth Endpoints](#)
    - [Register](#)
    - [Login](#)
    - [Refresh Token](#)
    - [Logout](#)
    - [Get Current User](#)
  4. [Image Endpoints](#)
    - [Upload Image](#)
    - [List My Images](#)
    - [Get Single Image](#)
    - [Serve Image File](#)
  5. [Google OAuth2 Login](#)
  6. [Error Reference](#)
  7. [Token Management Strategy](#)
  8. [Quick Test with cURL](#)
- 

## 1. Getting Started

### Base URL

`http://<server-ip>:8080`

During development with the server running locally:

`http://localhost:8080`

When deployed, replace with your server's public IP or domain.

### Content-Type

- All requests that send JSON: `Content-Type: application/json`
- Image upload: `Content-Type: multipart/form-data`
- All GET requests: no Content-Type header needed

### Authentication Header

Every endpoint except Register, Login, and Refresh requires:

`Authorization: Bearer <accessToken>`

The `accessToken` is returned when you register or log in.

---

## 2. Authentication Overview

HueVista uses a **dual-token system**:

| Token               | Lifetime   | Purpose                                                 |
|---------------------|------------|---------------------------------------------------------|
| accessToken (JWT)   | 15 minutes | Sent with every API request in the Authorization header |
| refreshToken (UUID) | 7 days     | Used once to get a new access token when it expires     |

### How to manage tokens in your mobile app

1. User registers or logs in
    - > Store accessToken and refreshToken securely (Keychain on iOS, EncryptedSharedPreferences on Android)
  2. Make API calls with:  
Authorization: Bearer <accessToken>
  3. When you get a 401 response:
    - > Call POST /api/auth/refresh with the refreshToken
    - > Store the new accessToken and refreshToken
    - > Retry the original request
  4. When the user logs out:
    - > Call POST /api/auth/logout
    - > Delete both tokens from local storage
- 

## 3. Auth Endpoints

### 3.1 Register

Create a new account with email and password.

POST /api/auth/register

Content-Type: application/json

Request body:

```
{
  "name": "Vikram Mali",
  "email": "vikram@example.com",
  "password": "MyPassword123"
}
```

**Field rules:** | Field | Required | Rules | | name | Yes | Cannot be blank | | email | Yes | Must be a valid email format | | password | Yes | Minimum 8 characters |

**Success response — 201 Created:**

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiI1NTBl...",
  "refreshToken": "550e8400-e29b-41d4-a716-446655440000",
  "tokenType": "Bearer",
  "expiresIn": 900,
  "user": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Vikram Mali",
    "email": "vikram@example.com",
    "picture": null,
    "provider": "LOCAL"
  }
}
```

**Field descriptions:** | Field | Type | Description | |—|—|—| | **accessToken** | string | JWT token. Send in Authorization header for all API calls. | | **refreshToken** | string | UUID. Store securely. Use to get a new accessToken when it expires. | | **tokenType** | string | Always “Bearer” | | **expiresIn** | number | Access token lifetime in seconds (900 = 15 minutes) | | **user.id** | string | UUID - the user’s unique identifier | | **user.name** | string | Display name | | **user.email** | string | Email address | | **user.picture** | string or null | Profile photo URL (null for email/password accounts) | | **user.provider** | string | “LOCAL” for email/password, “GOOGLE” for Google login |

#### Error responses:

```
// 400 - Email already registered
{
  "status": 400,
  "error": "Bad Request",
  "message": "Email already in use: vikram@example.com",
  "timestamp": "2026-05-12T10:30:00.000"
}

// 400 - Validation failed
{
  "status": 400,
  "error": "Bad Request",
  "message": "password: Password must be at least 8 characters",
  "timestamp": "2026-05-12T10:30:00.000"
}
```

### 3.2 Login

Log in with email and password.

POST /api/auth/login

Content-Type: application/json

#### Request body:

```
{
  "email": "vikram@example.com",
```

```
"password": "MyPassword123"
}
```

#### Success response — 200 OK:

Same shape as Register response above (AuthResponse).

#### Error responses:

```
// 401 - Wrong password or email not found
{
  "status": 401,
  "error": "Unauthorized",
  "message": "Bad credentials",
  "timestamp": "2026-05-12T10:30:00.000"
}
```

Note: The server intentionally returns the same error for “wrong password” and “email not found” to prevent attackers from finding out which emails are registered.

---

### 3.3 Refresh Token

Get a new access token using your refresh token. Call this when a request returns 401.

POST /api/auth/refresh

Content-Type: application/json

#### Request body:

```
{
  "refreshToken": "550e8400-e29b-41d4-a716-446655440000"
}
```

#### Success response — 200 OK:

Same shape as Login response (brand new accessToken + refreshToken pair).

IMPORTANT: The old refresh token is invalidated immediately after use. Always save the new refresh token from the response — do not reuse the old one.

#### Error responses:

```
// 400 - Refresh token expired (7 days passed)
{
  "status": 400,
  "error": "Bad Request",
  "message": "Refresh token expired. Please log in again.",
  "timestamp": "2026-05-12T10:30:00.000"
}

// 400 - Refresh token not found (already used or never existed)
{
  "status": 400,
  "error": "Bad Request",
```

```
"message": "Refresh token not found.",
"timestamp": "2026-05-12T10:30:00.000"
}
```

When either of these errors occurs: **redirect the user to the login screen.**

---

### 3.4 Logout

Revoke all tokens for the current user. Requires a valid access token.

POST /api/auth/logout

Authorization: Bearer <accessToken>

No request body.

Success response — 200 OK:

```
{
  "message": "Logged out successfully"
}
```

After this call: delete both tokens from local storage and navigate to the login screen.

---

### 3.5 Get Current User

Quick check to confirm the token is valid and get the logged-in user's ID.

GET /api/auth/me

Authorization: Bearer <accessToken>

Success response — 200 OK:

```
{
  "userId": "550e8400-e29b-41d4-a716-446655440000"
}
```

**Error response — 401:** Token is missing, expired, or invalid.

Use this on app launch to verify the stored token is still valid before showing the home screen.

---

## 4. Image Endpoints

All image endpoints require **Authorization: Bearer <accessToken>**.

### 4.1 Upload Image

Upload a photo of a room or house exterior. The server will: 1. Validate the file (type + size) 2. Run AI classification (Claude Vision) to confirm it is a house/room photo 3. Store the file 4. Return the image metadata

POST /api/images/upload

Authorization: Bearer <accessToken>

Content-Type: multipart/form-data

**Form field:** | Field | Type | Required | Description | |—|—|—|—| | **file** | File | Yes | The image file. JPEG, PNG, or WebP only. Max 10 MB. |

### How to send in mobile:

*Android (Kotlin / OkHttp):*

```
val requestBody = MultipartBody.Builder()
    .setType(MultipartBody.FORM)
    .addFormDataPart(
        "file",
        "image.jpg",
        File(imagePath).asRequestBody("image/jpeg".toMediaType())
    )
    .build()

val request = Request.Builder()
    .url("http://localhost:8080/api/images/upload")
    .addHeader("Authorization", "Bearer $accessToken")
    .post(requestBody)
    .build()
```

*iOS (Swift / URLSession):*

```
var request = URLRequest(url: URL(string: "http://localhost:8080/api/images/upload")!)
request.httpMethod = "POST"
request.setValue("Bearer \(accessToken)", forHTTPHeaderField: "Authorization")

let boundary = UUID().uuidString
request.setValue("multipart/form-data; boundary=\(boundary)", forHTTPHeaderField: "Content-Type")

var body = Data()
body.append("--\(\boundary)\r\n".data(using: .utf8)!)
body.append("Content-Disposition: form-data; name=\"file\"; filename=\"image.jpg\"\r\n".data(using: .utf8)!)
body.append("Content-Type: image/jpeg\r\n\r\n".data(using: .utf8)!)
body.append(imageData)
body.append("\r\n--\(\boundary)--\r\n".data(using: .utf8)!)
request.httpBody = body
```

### Success response — 201 Created:

```
{
  "imageId": "7f3d9a2e-1b4c-4e8f-a6d2-3c5e7b9f1a0d",
  "imageUrl": "/api/images/files/550e8400-e29b.../7f3d9a2e-1b4c.jpg",
  "originalFilename": "living-room.jpg",
  "imageType": "INDOOR",
  "fileSize": 2457600,
  "uploadedAt": "2026-05-12T03:07:00"
}
```

**Field descriptions:** | Field | Type | Description | |—|—|—| | **imageId** | string | UUID — use this to fetch or reference this image | | **imageUrl** | string | URL to fetch the raw image. For local storage: use with

base URL. For S3: full pre-signed URL valid 60 minutes. | | **originalFilename** | string | The filename from the upload | | **imageType** | string | “INDOOR” or “OUTDOOR” — determined by AI | | **fileSize** | number | File size in bytes | | **uploadedAt** | string | ISO 8601 datetime |

#### Error responses:

```
// 422 - Not a house/room image (selfie, food, car, etc.)
{
  "status": 422,
  "error": "Unprocessable Entity",
  "message": "Please upload a photo of an indoor room or outdoor house/building exterior. Selfies",
  "timestamp": "2026-05-12T10:30:00.000"
}

// 422 - Wrong file type
{
  "status": 422,
  "error": "Unprocessable Entity",
  "message": "Only JPEG, PNG, and WebP images are accepted.",
  "timestamp": "2026-05-12T10:30:00.000"
}

// 400 - File too large (over 10 MB)
{
  "status": 400,
  "error": "Bad Request",
  "message": "Maximum upload size exceeded.",
  "timestamp": "2026-05-12T10:30:00.000"
}

// 500 - AI service temporarily down
{
  "status": 500,
  "error": "Internal Server Error",
  "message": "Image classification service is temporarily unavailable. Please try again.",
  "timestamp": "2026-05-12T10:30:00.000"
}
```

---

## 4.2 List My Images

Get all images uploaded by the logged-in user, sorted newest first.

GET /api/images

Authorization: Bearer <accessToken>

No request body or query parameters.

Success response — 200 OK:

```
[
  {
```

```

    "imageId": "7f3d9a2e-1b4c-4e8f-a6d2-3c5e7b9f1a0d",
    "imageUrl": "/api/images/files/550e8400.../7f3d9a2e.jpg",
    "originalFilename": "living-room.jpg",
    "imageType": "INDOOR",
    "fileSize": 2457600,
    "uploadedAt": "2026-05-12T03:07:00"
  },
  {
    "imageId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "imageUrl": "/api/images/files/550e8400.../a1b2c3d4.jpg",
    "originalFilename": "front-of-house.jpg",
    "imageType": "OUTDOOR",
    "fileSize": 3145728,
    "uploadedAt": "2026-05-11T18:22:00"
  }
]

```

Returns an empty array [] if the user has not uploaded any images yet.

---

### 4.3 Get Single Image

Fetch metadata for one specific image.

GET /api/images/{imageId}

Authorization: Bearer <accessToken>

Replace {imageId} with the actual UUID, e.g.:

GET /api/images/7f3d9a2e-1b4c-4e8f-a6d2-3c5e7b9f1a0d

Authorization: Bearer <accessToken>

**Success response — 200 OK:**

Same shape as a single item in the List Images response.

**Error response — 404:**

```

{
  "status": 404,
  "error": "Not Found",
  "message": "Image not found: 7f3d9a2e-1b4c-4e8f-a6d2-3c5e7b9f1a0d",
  "timestamp": "2026-05-12T10:30:00.000"
}

```

Note: You get 404 whether the image doesn't exist OR it belongs to a different user. This is intentional — no information about other users' images is ever leaked.

---

### 4.4 Serve Image File

Download the raw binary image. Only needed when the server uses local storage.

If imageUrl in the response starts with /api/images/files/, prepend the base URL and call it:

```
GET /api/images/files/{userId}/{filename}
Authorization: Bearer <accessToken>
```

Example:

```
GET /api/images/files/550e8400-e29b-41d4-a716-446655440000/7f3d9a2e-1b4c.jpg
Authorization: Bearer <accessToken>
```

**Success response — 200 OK:** - Body: raw image bytes - Content-Type: image/jpeg, image/png, or image/webp

**Error response — 403 Forbidden:** Empty body. Returned if you try to access another user's image file.

When S3 is enabled on the server, `imageUrl` will be a full `https://...amazonaws.com/...` pre-signed URL instead. In that case, fetch the URL directly — no Authorization header needed, the signature in the URL is the authentication. The URL expires after 60 minutes.

### How to display the image in mobile:

```
// Check if it's a relative path (local storage) or full URL (S3)
if imageUrl.startsWith(with: "http") {
    // S3 presigned URL - fetch directly, no auth header
    loadImage(from: imageUrl)
} else {
    // Local storage - prepend base URL + add auth header
    loadImage(from: baseUrl + imageUrl, headers: ["Authorization": "Bearer \$(accessToken)"])
}
```

---

## 5. Google OAuth2 Login

Google login is a browser-based flow. The typical approach for mobile apps is to open a `WebView` or use the platform's `OAuth2` library.

### Flow

Step 1: Open in browser / `WebView`:

```
GET http://<server>:8080/oauth2/authorization/google
```

Step 2: User sees Google sign-in page, selects their account

Step 3: Google redirects back to the server at:

```
GET http://<server>:8080/login/oauth2/code/google?code=...
```

Step 4: Server processes the code, creates/updates the user account

Step 5: Server responds with `AuthResponse` JSON (same as login response)

-> Parse the `accessToken` and `refreshToken` from the response

-> Store them and proceed to the home screen

### Recommended mobile approach

Use your platform's `OAuth2` / `OpenID Connect` library pointed at the server's authorization URL:

http://<server>:8080/oauth2/authorization/google

The final response from the server will be the same `AuthResponse` JSON as the login endpoint.

## 6. Error Reference

### Standard error response shape

Every error from the API has this JSON structure:

```
{
  "status": 422,
  "error": "Unprocessable Entity",
  "message": "Human-readable description of what went wrong",
  "timestamp": "2026-05-12T10:30:00.000"
}
```

### HTTP status codes used

| Status                    | Meaning                        | When you'll see it                                           |
|---------------------------|--------------------------------|--------------------------------------------------------------|
| 200 OK                    | Success                        | GET requests, login, logout, refresh, me                     |
| 201 Created               | Resource created               | Register, image upload                                       |
| 400 Bad Request           | Invalid request                | Validation errors, bad email/password format, file too large |
| 401 Unauthorized          | Authentication failed          | Missing/invalid/expired token, wrong password                |
| 403 Forbidden             | Access denied                  | Trying to access another user's image                        |
| 404 Not Found             | Resource not found             | Image ID doesn't exist or belongs to another user            |
| 422 Unprocessable Entity  | Valid request, invalid content | Not a house/room image, wrong file type                      |
| 500 Internal Server Error | Server error                   | Claude API down, database error, storage error               |

### What to do for each status code

| Status | Action in your app                                                  |
|--------|---------------------------------------------------------------------|
| 400    | Show the <code>message</code> field to the user                     |
| 401    | Try refreshing the token. If refresh also fails, redirect to login. |
| 403    | Show "Access denied" — this shouldn't happen in normal app usage    |
| 404    | Show "Not found" or remove the item from the local list             |

| Status | Action in your app                                                                |
|--------|-----------------------------------------------------------------------------------|
| 422    | Show the <b>message</b> to the user — it explains what was wrong with their image |
| 500    | Show “Something went wrong, please try again” and log the error                   |

## 7. Token Management Strategy

Here is the complete token lifecycle for your mobile app:

### On app launch

1. Read `accessToken` and `refreshToken` from secure storage
2. If neither exists -> show Login screen
3. If tokens exist -> call `GET /api/auth/me`
  - 200 OK -> proceed to Home screen
  - 401 -> try refresh (step 4)
4. Try `POST /api/auth/refresh` with the `refreshToken`
  - 200 OK -> save new tokens, proceed to Home screen
  - 400 -> tokens are fully expired, show Login screen

### On every API request

1. Add `Authorization: Bearer <accessToken>` to the request
2. If response is 401:
  - a. Call `POST /api/auth/refresh`
  - b. If refresh succeeds: save new tokens, retry original request
  - c. If refresh fails (400): clear tokens, redirect to Login screen
3. If response is any other error: handle as described in Section 6

### Token storage

| Platform     | Recommended storage                           |
|--------------|-----------------------------------------------|
| Android      | EncryptedSharedPreferences (Jetpack Security) |
| iOS          | Keychain                                      |
| React Native | react-native-keychain or expo-secure-store    |
| Flutter      | flutter_secure_storage                        |

**Never store tokens in:** - Regular `SharedPreferences` (Android) — not encrypted - `UserDefaults` (iOS) — not encrypted - `AsyncStorage` (React Native) — not encrypted

## 8. Quick Test with cURL

Copy and paste these to test the API from your terminal.

## Register a new user

```
curl -X POST http://localhost:8080/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Test User",
    "email": "test@example.com",
    "password": "Test@1234"
  }'
```

## Login

```
curl -X POST http://localhost:8080/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{
    "email": "test@example.com",
    "password": "Test@1234"
  }'
```

Save the accessToken from the response:

```
ACCESS_TOKEN="eyJhbGciOi..."
```

## Check who is logged in

```
curl http://localhost:8080/api/auth/me \
  -H "Authorization: Bearer $ACCESS_TOKEN"
```

## Upload an image

```
curl -X POST http://localhost:8080/api/images/upload \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -F "file=@/path/to/your/room-photo.jpg"
```

## List all my images

```
curl http://localhost:8080/api/images \
  -H "Authorization: Bearer $ACCESS_TOKEN"
```

## Get one image by ID

```
curl http://localhost:8080/api/images/7f3d9a2e-1b4c-4e8f-a6d2-3c5e7b9f1a0d \
  -H "Authorization: Bearer $ACCESS_TOKEN"
```

## Refresh the access token

```
curl -X POST http://localhost:8080/api/auth/refresh \
  -H "Content-Type: application/json" \
  -d '{
```

```
"refreshToken": "550e8400-e29b-41d4-a716-446655440000"
}'
```

## Logout

```
curl -X POST http://localhost:8080/api/auth/logout \
-H "Authorization: Bearer $ACCESS_TOKEN"
```

---

## Summary: All Endpoints

| Method | Endpoint                     | Auth Required | Description                              |
|--------|------------------------------|---------------|------------------------------------------|
| POST   | /api/auth/register           | No            | Create account with name/email/password  |
| POST   | /api/auth/login              | No            | Login with email/password                |
| POST   | /api/auth/refresh            | No            | Get new tokens using refresh token       |
| POST   | /api/auth/logout             | Yes           | Revoke all tokens for current user       |
| GET    | /api/auth/me                 | Yes           | Get current user's ID                    |
| POST   | /api/images/upload           | Yes           | Upload + AI-classify + store image       |
| GET    | /api/images                  | Yes           | List all images (newest first)           |
| GET    | /api/images/{imageId}        | Yes           | Get single image metadata                |
| GET    | /api/images/files/**         | Yes           | Download raw image bytes (local storage) |
| GET    | /oauth2/authorization/google | No            | Start Google login (browser redirect)    |

---

*API Version: Phase 2 — Auth + Image Upload with Claude Vision AI classification Backend: Spring Boot 4.0.6 | Java 17 | PostgreSQL*